

PAHS 学习全集

****Zachary Zhou 专用 — 架构、流程、代码、Co-op 保底、大哥汇报****

日期 -> 2026 年 6 月
项目 -> PAHS `~/Desktop/PAHS`
配套 -> SMAS `~/Desktop/SMAS`, PIP `~/Desktop/PIP`
阶段 -> Gap year, 2026 年 9 月 SFU CS 入学, Co-op 准备

如何使用本 PDF

- 1. 第一遍：读 Part 1-3 (为什么做、大哥理论、生态)
- 2. 第二遍：读 Part 4-7 (架构、流程、该学的代码)
- 3. 第三遍：用 Part 10 自测题，合上电脑口头回答
- 4. 每周：对照 Part 11 学习单元 + 用 Part 12 模板向大哥汇报

Part 1 · 你的处境与学习目标

1.1 你是谁

- * 20 岁, Burnaby, Gap year
- * 2026 年 9 月 SFU Computer Science 入学
- * 已有项目：PAHS (主控)、SMAS (IG 图文)、PIP (短视频)、EduSync、Popboxx 相关
- * 工具：Cursor + DeepSeek API, 暂不用本地大模型

1.2 两条线，不打架

[竖杠 (Co-op 保底)] 课业、面试、能读能改代码 (合格：核心文件、Python、Git、LeetCode、系统设计基础)

这叫 T 型能力：有主轴能落地，有横杠能判断价值。

1.3 本项目你要学到什么程度

合格标准 (能口头回答即可)：

- 1. 用户一句话后，经过哪几步才出结果？
- 2. PAHS、SMAS、PIP 各自干什么？
- 3. Path A 和 Path B 区别？
- 4. 坏了先查哪一层？
- 5. 哪些能自动，哪些必须人审？
- 6. 能指出 6-8 个核心代码文件各自职责

不必：从零重写整个 PAHS；背每一行 AI 生成的代码。

Part 2 · 大哥 (Sam) 那句话怎么理解

2.1 原话三层

「真正要学的是能力能做多少；不需要学代码怎么写；因为已经没用了。」

方法 -> 别用旧办法死背语法当唯一能力
判断 -> 手写语法的稀缺性在下降，但判断力不会

2.2 他站在什么位置

项目负责人视角：能不能交付、能不能规模化、能不能帮 Popboxx/餐饮等场景干活。

他投资的是：真实痛点到可审核交付这条链，不是你会不会背 Python。

2.3 「能力能做多少」四个问法

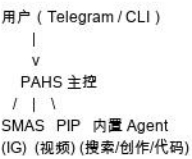
- 1. 范围 Scope：能从一句话做到预览图吗？能发帖吗？差在哪？
- 2. 稳定 Reliability：十次里几次成？挂在哪？
- 3. 速度 Speed：从需求到可给客户看要多久？
- 4. 边界 Boundary：什么绝不该承诺（开账号、付款、代发帖）？

2.4 映射到生活（简表）

PAHS -> 这周又多交付一件事，还是又多一个半成品？
餐厅工作 -> 今天多帮客人解决一件事了吗？
身体作息 -> 睡眠和运动是「能持续交付」的基础吗？

Part 3 · 项目生态

3.1 一张关系图（文字版）



3.2 各项目角色

[SMAS] IG 图文 + 预览图（合格：external_agents.yaml）
[PIP] 短视频生成（合格：external_agents.yaml）

PAHS 通过 subprocess 桥接调用 SMAS/PIP，不是把它们的代码拷进 PAHS。

3.3 两条执行路径

[过 Orchestrator] 是（合格：否）
[适合] 调研、多步、复杂任务（合格：快速出 IG/视频）
[人审] milestone + 总反馈（合格：SMAS：好 / 改：...）

学习时两条都要懂。产品体验靠 B；系统完整度和简历靠 A。

Part 4 · 架构分层

- [Orchestrator] 写 ExecutionPlan 任务表（合格：planning/orchestrator_planner.py）
- [Step Router] 计划结构校验（合格：planning/step_router.py）
- [Harness] 规则、预算、验证、能力边界（合格：harness/）
- [Agents] Searcher / Creator / Executor（合格：agents/）
- [Search Router] Tavily vs Perplexity（合格：agents/search_router.py）
- [External] SMAS / PIP 桥接（合格：external/）
- [Learner] 反馈到 rules / plan_patterns（合格：learning/）
- [Dev Lab] UI + 批量测试（合格：devlab/）

4.1 ExecutionPlan 结构

```
ExecutionPlan
intent_summary
complexity_band
review_policy
phases[]
phase: parallel?, depends_on
tasks[]: worker, tools, inputs_from
```

计划存 runs.plan_json，默认不展示给用户。预览：pah plan-preview "命令"

4.2 Harness 四层 + 能力边界

Tools -> 仅已批准工具

Validation -> verify_pipeline，失败可重试

Capability -> capability_brief：账号/发布/付款等诚实提示

Environment -> 预算、token、降级

Part 5 · LangGraph 完整流程 (Path A)

节点顺序 (src/pahs/graph/main.py)：

```
ingest
-> load_global_rules
-> triage_score
-> orchestrator_plan
-> plan_validate
-> env_precheck
-> load_target_rules
-> execute_plan_phase
-> verify_pipeline
-> present_milestone / final_present / retry / failed
-> milestone_human_review ( interrupt，等你 approved )
-> final_feedback_request ( interrupt，总反馈 )
-> Learner ( 总反馈后，提案需你 approve )
```

关键概念：

- * interrupt：图暂停，等你 pah reply 或 Dev Lab 里回复
- * checkpoint：SQLite 存状态，可 resume
- * BLOCKED：预算或环境检查不通过（批量测试已修每轮隔离预算）

Part 6 · 三套路由

【2】

序号：2

机制：Step Router

何时：plan 后

决定什么：计划是否合法

状态：结构校验

【3】

序号：3

机制：Search Router

何时：Searcher 执行

决定什么：Tavily vs Perplexity

状态：已完成

Search 两步：Step1 搜索 API 拿资料；Step2 DeepSeek 写最终报告。

Part 7 · 值得学的代码（按优先级）

7.1 必读（架构 + Co-op 能讲能改）

[P0] graph/runner.py（合格：start_run、resume_run）

[P0] planning/orchestrator_planner.py（合格：怎么生成计划）

[P0] planning/plan_executor.py（合格：phase 执行、并行、artifacts）

[P1] external/smas_bridge.py（合格：怎么调 SMAS）

[P1] external/pip_bridge.py（合格：怎么调 PIP、mock、超时）

[P1] harness/capability_brief.py（合格：能力缺口检测）

[P1] gateway/telegram_adapter.py（合格：Telegram 入口）

[P2] agents/searcher.py (合格：两步搜索)
[P2] agents/search_router.py (合格：搜索路由)
[P2] devlab/batch_runner.py (合格：批量测试怎么跑)
[P2] learning/learner.py (合格：提案怎么生成)
[P2] learning/batch_learner.py (合格：批量后整体分析)

7.2 配置必读

config/budget.yaml -> 预算限制
config/dev_batch_scenarios.yaml -> 批量测试 15 场景
.env -> API keys (勿提交 git)

7.3 Co-op 保底 (不在本 repo , 要并行)

- * 数据结构 + 算法 (LeetCode 中等)
- * Git、HTTP、REST、SQL 基础
- * 能读 Python、会 debug、会写小函数
- * 系统设计：能画「用户-API-DB」三层图

PAHS 是应用层实战；LeetCode + 课业是通行证。

Part 8 · Dev Lab 与批量测试

8.1 两种方式

[批量测试] pah dev-batch (合格：自动跑 N 次，出报告)

8.2 批量测试流程

dev_batch_scenarios.yaml
-> pah dev-batch --runs N --mock
-> 每轮 auto approve + synthetic feedback
-> dev_batch_report_*.md (成绩单)
-> dev_batch_improvement_plan_*.md (改进方案)
-> 你：Handoff 给 Cursor 改代码；approve 有用提案

8.3 Mock vs 真实 API

【真实】

模式：真实
命令：--no-mock
测什么：DeepSeek 文案与计划质量
建议次数：5-10

Mock 100/100 只说明流程稳，不代表 AI 质量合格。

8.4 清理

pah reset-test (输入 DELETE ALL) 清理 runs、提案、报告。

data/ 和 rules/learnings/*.json 已在 .gitignore。

Part 9 · 常用命令

pah pending / pah reply -> 审核
pah feedback "..." -> 总反馈
pah proposals pending -> 看 Learner 提案
pah proposals approve id -> 批准提案
pah dev-ui -> Dev Lab 网页
pah dev-batch --runs 100 --mock -> 批量 mock 测试

```
pah dev-batch --runs 10 --no-mock -> 真实 API 小批量
pah plan-preview "..." -> 预览计划
pah search-status -> 搜索配置
pah telegram -> 启动 bot
pah reset-test -> 清理测试数据
```

注意：python3 -m pabs.cli 或 venv 里 pah，不是 python3 pabs.cli

Part 10 · 自测题库 (合上电脑回答)

流程 6 问

1. Telegram 说「做 IG 图文」走哪条路径？经过哪些模块？
2. pah run 调研任务和直调 SMAS 有何不同？
3. milestone_review 和 final_feedback 区别？
4. Learner 提案为何不会自动生效？
5. capability_brief 检测到开账号任务时会怎样？
6. dev-batch 里 BLOCKED 曾经什么原因？现在怎么修的？

代码 4 问

1. runner.py 里 resume 做什么？
2. plan_executor 里 parallel 什么意思？
3. pip_bridge 在 dev-batch mock 时有什么不同？
4. batch_learner 和单次 learner 区别？

合格：每题能答 3 句话以上。

Part 11 · 四周学习单元

第 1 周：心智模型 + Path B

- * 读 Part 1-3、第五章 Telegram 走读 (详细版教程)
- * 动手：pah telegram，发一条 IG 需求
- * 自测：画出 Telegram 到 SMAS 的数据流

第 2 周：Path A + LangGraph

- * 精读 graph/main.py、runner.py
- * 动手：pah run + pah reply approved + feedback
- * 自测：列出 10 个节点名

第 3 周：Orchestrator + External

- * 精读 planning/、external/
- * 动手：pah plan-preview；改 external_agents.yaml 一个关键词
- * 自测：解释 ExecutionPlan 三字段

第 4 周：Harness + 测试 + 汇报

- * 精读 capability_brief、batch_runner、batch_learner
- * 动手：dev-batch --runs 10 --mock；读 improvement_plan
- * 自测：向 Sam 做 5 分钟 demo 讲解

并行：每周 3-5 道 LeetCode；SFU 预习课若有则跟进。

Part 12 · 向大哥 (Sam) 汇报模板

12.1 每周三段式 (微信/语音均可)

****本周交付 (结果)****

- * 做成了什么客户能感知的事 (例 : Telegram 能出 IG 预览 ; 批量测试 100 次全过)

****能力边界 (诚实)****

- * 现在能做到哪一步 ; 还做不到什么 (例 : 不能代发帖、不能开平台账号)

****下周一件事 (可验证)****

- * 只承诺一件具体交付 , 不要列 10 个大计划

12.2 他爱听的词汇

- * 稳定、可审核、可复用、省人力、开业引流、预览给客户看
- * 少讲 : 我学了很多代码、我改了 50 个文件

12.3 避免的汇报方式

- * 只讲过程不讲结果
- * 承诺全自动发帖/开账号 (系统故意不做)
- * 一堆技术名词没有 demo

Part 13 · 设计原则 (背下来能讲)

1. Orchestrator 发任务表 , Agent 按表执行
2. 计划内部存储 , 默认不展示
3. 阶段内可并行 , artifacts 下传
4. 每个 Phase 过 Harness
5. 三层路由 : Triage、Step Router、Search Router
6. Learner 永不自动改生产规则
7. 你定义「完成」 : milestone + 总反馈
8. 诚实能力边界 : 账号、支付、直发
9. Mock 测流程 , 真实 API 小批测质量
10. Builder 工具永不自动上线

Part 14 · 相关文档索引

- * docs/PAHS_深度学习教程_详细版.md (代码走读)
- * docs/PAHS_架构流程图.md
- * docs/ARCHITECTURE.md
- * docs/EXTERNAL_AGENTS_SETUP.md
- * docs/DEVLAB_ROADMAP.md
- * README.md (最新总览)

附录 · 术语中英对照

- 任务表 -> ExecutionPlan
- 阶段审核 -> Milestone review
- 总反馈 -> Final feedback
- 提案 -> Proposal
- 能力缺口 -> Capability gap
- 外部工具 -> External agent
- 批量测试 -> dev-batch

PAHS 学习全集 · Zachary Zhou · 2026-06

下卷 · 深度学习教程 (代码走读详细版)

****给 Zachary Zhou 的自学用书 — 不是清单，是逐段解释****

配套项目 -> SMAS `~/Desktop/SMAS`、PIP `~/Desktop/PIP`

版本 -> 2026-06 详细版 v2

怎么用 -> 按章顺序读；每章末尾做「动手」和「自测」

写在前面：这份教程和上一份有什么不同

上一份手册像「地图」——告诉你有什么山、什么河。

****这一份像「导游讲解」——带你一步一步走，解释路上每一块石头是什么、为什么放在这里。**

学习时请记住你大哥 (Sam) 的那句话应该怎么理解：

- * ****要学****：这套系统****能稳定做到哪一步****、****为什么这样设计****、****坏了怎么查****
- * ****不必主攻****：从零默写每一行 Python 语法
- * ****但必须能****：合上电脑，用自己的话讲清楚「用户发一句话后发生了什么」

第一章 · 这个项目到底是什么 (建立心智模型)

1.1 用一句话定义 PAHS

****PAHS (Personal Agent Harness System) = 你的私人 AI 总调度台。****

它不像 ChatGPT 网页那样「你问一句它答一句」就结束。它更像一个****项目经理****：

1. 听懂你要什么 (Telegram / CLI)
2. 判断这事该谁干 (自己写？搜索？调 SMAS 做图？调 PIP 做视频？)
3. 盯着干完 (调用工具、检查输出)
4. 把结果给你看，****等你点头**** (重要内容必须人审)
5. (可选) 记住你的偏好，下次做得更好

1.2 三个项目的关系 (一定要记牢)

很多人第一次学时会晕：****PAHS、SMAS、PIP 到底是三个东西还是一个？****

[****SMAS****] 设计部 (专做 IG 图文) (合格：独立 Python 项目，有自己的 AI 流水线)

[****PIP****] 视频部 (专做短视频) (合格：同上)

PAHS ****不代替**** SMAS 生成图片。PAHS 只做一件事：****用 subprocess 调用 `~/Desktop/SMAS/scripts/smas.sh generate "你的需求"`，然后把结果转成 Telegram 能发的图片和文字。****

这叫 ****External Agent Bridge (外部工具桥梁)****。好处是：

- * SMAS 可以单独开发、单独测试
- * 以后加「建站工具」「发邮件工具」只需改 `config/external\_agents.yaml`
- * PAHS 主控代码保持薄

1.3 你日常用的路径 vs 课本里的路径

你现在 Telegram 上主要走的是 ****「直调路径」****：

Telegram 消息 -> PAHS 认意图 -> 直接调 SMAS -> 回图

PAHS 代码里还有一条 ****「LangGraph 全流程」**** (`pah run "..."`)：

命令 -> 分流 Triage -> 编排计划 -> Creator/Searcher 干活 -> 多轮审核 -> 结束反馈 -> Learner

****为什么要两条？****

- * ****直调****：快、像真人助理，适合「给咖啡店做 IG」这种单一任务
- * ****全流程****：复杂、可暂停恢复，适合调研、写长文、写代码、多里程碑

学习时两条都要懂。你现在的产品体验靠直调；你 co-op 简历和系统完整度靠 LangGraph。

第二章 · 实战走读：一条 Telegram 消息的完整一生

这一章是最核心的。请对着代码读。

****假设你在 Telegram 对 bot 说：****

> 给咖啡店做一条开业 IG 图文

2.1 第一步：终端里谁在监听？

你运行：

```
cd ~/Desktop/PAHS
source .venv/bin/activate
pah telegram
```

`pah telegram` 调用 `telegram\_adapter.py` 里的 `run\_telegram\_bot()`，内部是：

```
app.run_polling()
```

****polling 的意思****：Python 进程每隔一小段时间问 Telegram 服务器：「有新消息吗？」

所以终端「停住不动」= ****正常****，不是卡死。这个进程必须一直开着，bot 才在线。

****学习点****：任何「聊天机器人」都需要一个****常驻进程****。以后上云，就是把这段跑在 VPS/Fly.io 上，而不是你笔记本上。

2.2 第二步：Telegram 把消息交给谁？

文件：`src/pahs/gateway/telegram\_adapter.py`

```
async def on_message(update, context):
    await _process_message(update, raw_text=update.message.text)
```

`\_process\_message` 做三件事：

(1) 取出 chat_id

```
chat_id = str(update.effective_chat.id)
```

这是 Telegram 给你的对话唯一 ID。以后「这个聊天是否在等 SMAS 审核」就靠它查 session。

(2) 规范化用户输入

```
normalized = normalize_telegram_input(raw_text)
```

`persona.py` 里：如果你发 `/generate 咖啡店开业`，会转成 `做一条 IG 图文：咖啡店开业`。

目的是让后面路由规则更简单。

(3) 如果像是要调工具，先回一句「人话」

```
tool = infer_external_agent(normalized)
if tool is not None:
    await update.message.reply_text(friendly_working(tool.name))
    # -> "好，我先帮你做 IG 图文，可能要一两分钟，稍等~"
```

****产品细节****：先回一句，用户不会以为 bot 死了。生成可能要 1-3 分钟（要调 DeepSeek、fal 生图）。

(4) 交给统一大脑

```
payload = handle_inbound_text(
    normalized,
    channel="telegram",
    channel_user_id=chat_id,
    normalized=True,
)
```

****学习点****：Telegram 适配器很薄。真正逻辑在 `service.py` 的 `handle\_inbound\_text`——这样以后加 WhatsApp 不用重写业务。

2.3 第三步：`handle\_inbound\_text` 怎么决策？

文件：`src/pahs/gateway/service.py`

函数按 ****优先级从上到下**** 判断（这段顺序极其重要）：

判断 1：是不是 `reply run\_xxx ...` ？

CLI 跨渠道审核会用。Telegram 直调模式你一般不用。

判断 2：是不是查 `pending` / `status` ？

运维命令。

判断 3：Telegram 直调模式（你当前的主路径）

```
if channel == "telegram" and _telegram_direct_tools():
```

读 `config/gateway.yaml`：

```
telegram_direct_tools: true
```

3a. 你是不是在回复上一张预览图？

```
if is_smas_review_reply(stripped) and get_session(channel_user_id):  
    return execute_smas_review(...)
```

`telegram\_session.py` 里，`好` / `ok` / `改`：字大一点` 会识别为审核回复。

但****只有**** `data/telegram\_sessions.json` 里记录「这个 chat 正在等 SMAS 审核」时才生效。

3b. 是不是新任务，要调外部工具？

```
tool = infer_external_agent(stripped)  
if tool is not None:  
    return execute_direct_tool(tool.name, stripped, ...)
```

「给咖啡店做开业 IG」-> 命中 `smas`（下一章细讲）。

判断 4：是不是 `run ...` 或 `@smas` 显式命令？

走 LangGraph `start\_run`。

判断 5：闲聊

```
return _handle_telegram_chat(stripped)
```

先匹配 `quick\_reply`（「在吗」「看得到吗」），否则调 DeepSeek 用 `PAHS\_PERSONA\_SYSTEM` 人格回复。

2.4 第四步：意图路由——为什么知道要调 SMAS？

文件：`src/pahs/gateway/intent\_router.py`

```
def infer_external_agent(text: str) -> ExternalAgentSpec | None:
```

两层匹配：

****第一层：显式前缀****（`registry.py` 的 `match\_external\_agent`）

```
* `@smas 做图`  
* `smas: 做图`
```

****第二层：自然语言关键词****（`INTENT\_RULES`）

```
("smas", ["Instagram", "ins post", "发帖", "图文", "做一条图文", ...])
```

你的句子「给咖啡店做一条开业 ****IG 图文****」——

注意：`IG` 单独不在列表里，但 ****「图文」**** 在。所以会返回 SMAS。

****如果路由失败会怎样？****

`infer\_external\_agent` 返回 `None` -> 不会调 SMAS -> 掉进闲聊或 `friendly\_help`。

****学习时的调试命令：****

```
pah route-preview "给咖啡店做一条开业 IG 图文"
```

2.5 第五步：`execute\_direct\_tool` 里发生什么？

文件：`src/paahs/gateway/direct\_tools.py`

5.1 创建 run 记录

```
run_id = new_run_id() # 例如 run_20260624_153045_a3f2
db.create_run(run_id, prompt, channel="telegram", status="ACTIVE")
```

即使直调模式不走 LangGraph，**仍然记账**。以后查历史、接 Learner 都靠这个。

5.2 调用外部工具

```
result = run_external_agent(agent_name, prompt, run_id=run_id)
```

`runner.py` 根据 type 分发到 `run\_smas()` 或 `run\_pip()`。

5.3 格式化交付内容

SMAS 专用：

```
delivery_text, image_path = _format_smas_delivery(result)
```

- * 从 result 里拿 `preview\_image` 路径
- * 文案用 `result["text"]`（已在 bridge 里从 caption.json 格式化过）
- * 若有图，追加审核脚：

满意回复：好

要修改回复：改：你的修改意见

5.4 设置「等待审核」状态

```
if agent_name == "smas" and image_path and channel == "telegram":
    status = "AWAITING_REVIEW"
    set_smas_review(channel_user_id, run_id=run_id, image_path=image_path)
```

`telegram\_sessions.json` 示例：

```
{
  "123456789": {
    "tool": "smas",
    "run_id": "run_20260624_153045_a3f2",
    "status": "awaiting_review"
  }
}
```

5.5 返回给 Telegram 适配器

```
return {
  "action": "deliver",
  "text": delivery_text,
  "image_path": image_path,
  "awaiting_review": True,
  ...
}
```

2.6 第六步：Telegram 怎么把图发给你？

回到 `telegram\_adapter.py`：

```
if action == "deliver":
    intro = friendly_delivery_intro("smas", awaiting_review=True)
    # -> "预览图来了（SMAS），你看下这版："
    body = intro + "\n\n" + delivery_text

    if image_path and Path(image_path).is_file():
        await update.message.reply_photo(photo=handle, caption=body[:1024])
```

****学习点**：**

- * Telegram 图片 caption 最长 1024 字符，超长要 `reply\_text` 发剩余
- * 图片是 **本地文件路径**，不是 URL——PAHS 读盘上传给 Telegram

2.7 第七步：你回复「好」或「改：...」

再发「好」-> `is\_smas\_review\_reply` 为真 -> `execute\_smas\_review`：

```
action, payload = parse_smas_review_reply("好") # -> ("approve", "好")
smas_text = "ok"
result = run_smas_action(spec, "ok") # 调用 smas.sh message ok
```

「改：标题字大一点」-> `smas\_text` = "edit: 标题字大一点" -> SMAS 内部走 edit 流程。

2.8 本章自测（合上电脑回答）

1. 为什么 `pah telegram` 终端看起来「卡住」？
2. `handle\_inbound\_text` 在调 SMAS 之前检查哪几件事？
3. `run\_id` 在直调模式里有什么用？
4. 预览图存在你电脑的什么位置？（提示：SMAS state 目录）
5. 若用户说「你好」会不会调 SMAS？走哪条逻辑？

第三章 · SMAS Bridge 深度讲解：PAHS 如何与 SMAS 对话

文件：`src/pahs/external/smas\_bridge.py`

PAHS 和 SMAS 之间 **没有 import 关系**，只有 **命令行 + JSON**：

~/Desktop/SMAS/scripts/smas.sh generate "给咖啡店做一条开业 IG 图文"

stdout 是一坨 JSON，PAHS 解析它。

3.1 为什么要 `generate` 而不是 `message`？

SMAS 有两个入口（`openclaw\_bridge.py`）：

[message <文本>] Orchestrator().handle_text(text)（合格：对话、审核 ok/skip、改图）

`handle\_text` 里有意图路由器。若它听不懂，会返回 `\_help\_message()`——就是你看过的那堆 `generate` 说明书。

所以 PAHS 直调必须用 `generate`，配置在：

```
# config/external_agents.yaml
smas:
  subcommand: generate
```

3.2 `clean\_smas\_prompt` 干什么？

用户有时说：「帮我做 IG post **并且告诉我调用了什么 agent**」

这句话里的后半句是**问 PAHS 的**，不是给 SMAS 的创作需求。

`clean\_smas\_prompt` 用正则删掉这类尾巴，避免 SMAS 困惑。

3.3 `\_clear\_smas\_pending` — 为什么要先 skip？

SMAS 内部有状态机（`~/Desktop/SMAS/state/state.json`）。

若上一次生成完后你没点 ok/skip，状态是：

```
{ "status": "waiting_review", ... }
```

此时再 `generate`，SMAS 会拒绝：

> The previous content is still waiting for review.

PAHS 在生成前自动执行：

```
smas.sh message skip
```

学习点：两个系统都有状态。PAHS 的 session 管「Telegram 是否在等人审」；SMAS 的 state 管「内容流水线是否结束」。桥接层要处理两边。

3.4 `\_parse\_bridge\_output` 和 `\_finalize\_smas\_payload`

SMAS 返回的 JSON 里有：

```
{
  "ok": true,
  "text": "Preview ready. Please review: ... Reply ok / publish ...",
  "preview_image": "/Users/.../SMAS/state/preview_feed.png",
  "caption_file": ".../caption.json"
}
```

PAHS **故意不用** `text` 里的说明书，而是：

1. 读 `caption.json` 拼 Hook / 正文 / Hashtags (`\_format\_smas\_text`)
2. 过滤「Generation failed」「waiting for review」-> 友好中文 (`\_smas\_failure\_message`)
3. 判断 `ok`：必须有真实存在的 `preview\_image` 才算成功

为什么？ 因为 SMAS 的 `text` 是给「直接用 SMAS CLI 的人」看的；PAHS 要给 Telegram 用户看**产品级文案**。

3.5 动手：自己跑一遍 bridge

```
cd ~/Desktop/PAHS && source .venv/bin/activate
pah externals test smas "给咖啡店做一条开业 IG 图文"
```

观察输出字段：

- * `command`：实际执行的 shell 命令
- * `preview\_image`：有没有路径
- * `text`：PAHS 格式化后的文案
- * `ok`：最终是否算成功

再打开：

```
cat ~/Desktop/SMAS/state/state.json
cat ~/Desktop/SMAS/state/caption.json
ls -la ~/Desktop/SMAS/state/preview_feed.png
```

建立直觉：生成 = 改一堆 state 文件 + 出一张 png。

第四章 · SMAS 内部：生成一张 IG 图要经过什么

你在 PAHS 项目里学习，也必须懂 SMAS 在桥接层后面干什么，否则报错你看不懂。

文件：`~/Desktop/SMAS/core/orchestrator.py`

4.1 `generate(user\_request)` 流程

1. 检查 state.json — 是否 waiting_review / confirm_post_type
2. ContentPipeline().run_guided(user_request)
3. 读 caption.json
4. build_review_prompt(caption) + preview_path 返回

4.2 `run\_guided` 大致阶段

(名称在 `content\_pipeline.py`，你学习时知道顺序即可)

```
classify ( 分类帖子类型 )
-> brief ( 创意简报 )
-> caption ( 文案 hook/body/hashtags )
-> visual_director ( 选 Path A/B/C、视觉参数 )
-> image ( 调 fal 等 API 生图 )
-> preview ( 合成 IG 预览框图 )
-> critic ( AI 打分建议 )
-> 状态变为 waiting_review
```

4.3 Path A / B / C 是什么？

[B] 用产品参考图 + fal edit 合成场景 (合格：有产品资产)

[C] 模板 + 文字叠层 Pillow (合格：开业、活动海报类)

「咖啡店开业」常走 `**Path C**`，需要 ``text_overlay.lines`` 是结构化对象。
你遇到过 Pydantic 报错，就是因为 LLM 返回了字符串列表 ``["NOW OPEN", ...]`` 而不是 ``[{"text": "NOW OPEN", "zone": "top-left", ...}]``。
PAHS/SMAS 侧已加归一化，这是 `**LLM 输出不可靠 -> 工程要清洗**` 的典型案例。

第五章 · LangGraph 全流程 (`pah run`) 详解

当你执行：

```
pah run "调研 LangGraph checkpoint 并写中文笔记"
```

5.1 `start\_run` 做什么？

文件：``src/pahs/graph/runner.py``

```
graph = build_graph()
config = graph_config(run_id) # thread_id = run_id, 用于 checkpoint
initial_state = {"run_id": run_id, "user_command": command, "channel": channel}
result = graph.invoke(initial_state, config=config)
```

`**invoke**` = 从图入口跑到遇到 ``interrupt`` 或 ``END``。

5.2 图的主干 (`graph/main.py`)

用白话讲每个节点：

```
load_global_rules -> 加载全局规则 ( 安全、预算 )
triage_score -> 判断任务简单还是复杂
orchestrator_plan -> 出计划：用哪个 worker、几个里程碑
env_precheck -> 检查预算、API、是否降级模型
load_target_rules -> 加载该 worker 专属规则
worker_execute -> 真干活：Creator / Searcher / Executor / External
verify_pipeline -> 校验输出质量
present_milestone -> 把结果呈现给你
milestone_human_review -> **interrupt — 暂停等你输入**
final_present -> 最终呈现
final_feedback_request -> **再 interrupt — 要最终反馈**
```

5.3 什么是 `interrupt` ？

LangGraph 里：

```
feedback = interrupt(("type": "milestone_review", "presentation": "..."))
```

程序在这里`**冻结**`，状态存进 checkpoint (SQLite)。

你执行：

```
pah reply run_xxx "approved"
```

``resume_run`` 用 ``Command(resume=user_input)`` 从冻结处继续。

`**和直调的区别**`：LangGraph 路径`**天生为多轮审核设计**`；直调是为`**一次交付**`优化。

5.4 完成后 Learner 怎么触发？

``resume_run`` 末尾：

```
if not interrupts and result.get("status") == "COMPLETED":
    proposals = learn_from_final_feedback(run_id, feedback)
```

`**注意**`：Telegram 直调 SMAS `**目前不会自动走这里**`。这是你项目「还没接满」的重要一点。

第六章 · Harness 四层：为什么叫 Harness (挽具)

Harness 原意是套在马身上控制方向的挽具。

****PAHS 的 Harness = 套在 AI 外面，防止它乱跑。****

6.1 Rule Layer (规则层)

文件：`harness/rules.py`、`rules/` 目录

****问题****：如果每次把全部公司规定塞给 AI，prompt 会爆、成本高、注意力散。

****做法****：全局规则每次加载；具体到 Creator 的规则只有选中 Creator 时才加载。

Learner 批准后，内容 append 到 `standards/user\_preferences.md` 等，下次 `standards\_loader.py` 会读进来。

6.2 Tool Layer (工具层)

文件：`harness/tools.py`、`external/registry.py`

****原则****：Orchestrator 只能调用****注册过****的工具。

Builder 在 `tools/staging/` 生成的工具，****不会****出现在生产列表里，直到你 `pah tools approve`。

6.3 Validation Layer (校验层)

文件：`harness/validation.py`

多层：确定性检查 -> 启发式 -> 必要时 LLM 再审 -> 最后才是人审。

****省钱****：不是所有输出都要再问一次 GPT 「好不好」。

6.4 Environment Layer (环境层)

文件：`harness/environment.py`、`harness/budget.py`

管：token 预算、超时、模型降级、API 挂了怎么办。

****商业意识****：大哥会问「跑一条多少钱」——答案在这层记。

第七章 · 数据存在哪：SQLite 与文件

7.1 `~/Desktop/PAHS/data/runs.db`

表 (见 `storage/schema.sql`)：

- * ****runs****：每次任务一条，含 status、command、channel
- * ****review_queue****：LangGraph 路径等人审的队列
- * ****events****：日志流，`pah events <run\_id>` 可看

7.2 `data/telegram\_sessions.json`

直调 SMAS 后「等用户说好/改」的临时.session，不是长期偏好库。

7.3 SMAS 的 `state/` 目录

一次生成的「工作台」：caption、visual_spec、preview 图、critic 报告全在这。

****学习练习****：生成一次后，逐个打开 json 文件，看字段含义。

第八章 · Learner：反馈怎么变成系统记忆

文件：`learning/learner.py`

8.1 设计哲学

- * ****里程碑审核**** (「这版大纲不行」) -> 只影响****这一次****任务
- * ****最终反馈**** (「以后都要更口语」) -> 可以变成****永久提案****

且永久提案默认 ****pending****，要你：

pah proposals approve <id>

才会写入 `standards/` 或 `rules/`。

8.2 为什么不让 AI 自动改自己？

想象 AI 学歪了：「用户骂了一句」-> 系统永久改成「永远只写一句话」-> 以后全毁。

****人工批准****是安全阀。

8.3 现状诚实说明

`analyze\_feedback` 目前是****关键词规则**** (Mock)，不是 DeepSeek 深度分析。

Telegram 里「改：标题大一点」****还没有****自动进 Learner——这是你下一步开发的好课题。

第九章 · 配置文件逐行读懂

9.1 `.env`

```
TELEGRAM_BOT_TOKEN=123456:AAFxxxx # 必须含一个冒号，BotFather 全长
DEEPSEEK_API_KEY=sk-xxxx
PAHS_LLM_PROVIDER=auto
```

9.2 `config/external\_agents.yaml`

每个外部工具一段：

- * `enabled`：开关
- * `project\_dir`：SMAS/PIP 根目录
- * `subcommand`：SMAS 用 generate
- * `match\_keywords`：自然语言路由
- * `timeout\_seconds`：防止生图卡死

9.3 `config/gateway.yaml`

```
telegram_direct_tools: true # false 则 Telegram 只闲聊/走 run
telegram_chat_fallback: true # 无工具匹配时是否 DeepSeek 闲聊
```

第十章 · 故障排查：像工程师一样分层

- [InvalidToken] 配置 (合格：`.env` token 是否完整)
- [返回命令说明书] SMAS 子命令 (合格：是否误用 message；看 smas_bridge)
- [无图只有字] SMAS 生成 (合格：pah externals test smas；看 state.json error)
- [waiting for review] SMAS 状态 (合格：state.json；是否需 skip)
- [Pydantic 报错] SMAS visual (合格：LLM 输出格式；归一化是否生效)
- [Connection error] API/网络 (合格：SMAS .env 里 fal/DeepSeek key)

****口诀****：用户看到的 -> PAHS -> bridge 命令 -> SMAS 状态文件 -> 外部 API。

第十一章 · 和大哥 (Sam) 的话怎么对齐

大哥说学「能力能做多少」——用本教程，你应该能答：

****现在能做****：

- * Telegram 自然语言 -> IG 预览图 + 分块文案 -> 人审 ok/改

****现在不能做****：

- * 自动发帖、自动开号、全自动建站上线

****技术为什么可信****：

- * Harness 有人审、有状态、有桥接隔离，不是裸 ChatGPT

****你学的不是语法，是：****

- * 第二章的完整链路 (Telegram 一条消息从头到尾)
- * 第三章的 bridge 契约 (PAHS 和 SMAS 怎么说话)
- * 第六章为什么直调和 LangGraph 并存

第十二章 · 12 周学习作业 (每周可检验)

- [2] 跑 externals test smas 并读 4 个 state 文件 (合格：能说出每个文件作用)
- [3] 改 INTENT_RULES 加一个关键词 (合格：Telegram 新句式可路由)
- [4] 走通一次 pah run 加 pah reply (合格：理解 interrupt)
- [5] 读 graph/main.py 列出所有节点 (合格：能对应 Harness)
- [6] 创建并 approve 一条 proposal (合格：文件出现在 standards 目录)
- [7] 写 1 页给 Sam 的能力边界 (合格：他看得懂、无夸大)
- [8] 录 60 秒 demo 视频 (合格：开业 IG 全链路)
- [9] 讲清 Harness 四层各举一个例子 (合格：不看笔记能说完)
- [10] 独立排查一次生成失败 (合格：能说出是哪一层坏了)
- [11] 把 Learner 接到 Telegram 改：反馈 (合格：能生成 proposal)
- [12] 整理 co-op 用项目介绍稿 (合格：英文 2 分钟能讲完)

附录 · 关键文件与函数索引

总调度决策 -> gateway/service.py` -> `handle_inbound_text
自然语言路由 -> gateway/intent_router.py
直调交付 -> gateway/direct_tools.py
SMAS 桥接 -> external/smas_bridge.py
外部工具注册 -> config/external_agents.yaml
LangGraph 图 -> graph/main.py` -> `build_graph
启动/恢复 run -> graph/runner.py
学习闭环 -> learning/learner.py
SMAS 生成核心 -> ~/Desktop/SMAS/core/orchestrator.py

****教程结束。建议：打印本章自测题，每周重做一遍，直到不用看代码也能答。****